Vault Access - Challenge 1

Vulnerability Identified

SQL Injection in the employee portal login at /employee-portal. The username field is directly concatenated into a SQL query without any sanitization or parameterization, allowing an attacker to inject arbitrary SQL and bypass the password check entirely.

Steps Followed

1. The challenge said to access the employee dashboard without valid credentials. Found the employee portal at /employee-portal, separate from the regular customer login.
2. The hint said "the difference between wrong password and welcome back is decided by more than the password field." This screamed SQL injection — the database query probably checks both fields, and I could manipulate one to ignore the other.
3. In the username field, I typed: admin' – The ' closes the username string in the SQL query, and -- comments out everything after it (the password check). So the query becomes "find user where username is admin" and ignores the password completely.
4. Left the password field as anything (typed blah) and hit submit. Got straight in.
5. The employee dashboard loaded with the vault flag displayed right on the page.
6. The whole query the server ran was something like: SELECT * FROM employees WHERE username='admin' --' AND password='blah' — everything after -- is just a comment, so the password never mattered.

Flag Obtained - FLAG{vault_0d5a9a7b}

Impact

In a real bank, an employee portal without proper SQL safeguards means anyone could access internal dashboards, HR records, salary data, and administrative tools. The same SQL injection could also be used to dump entire database tables or modify records — it's not just a login bypass, it's full database access.

Mitigation / Remediation

Don't build SQL queries by gluing strings together. Use parameterized queries (the ? placeholder approach) so user input is always treated as data, not code. The regular customer login on the same app already does this correctly — the employee portal just didn't follow the same pattern.

Search & Steal - Challenge 4

Vulnerability Identified

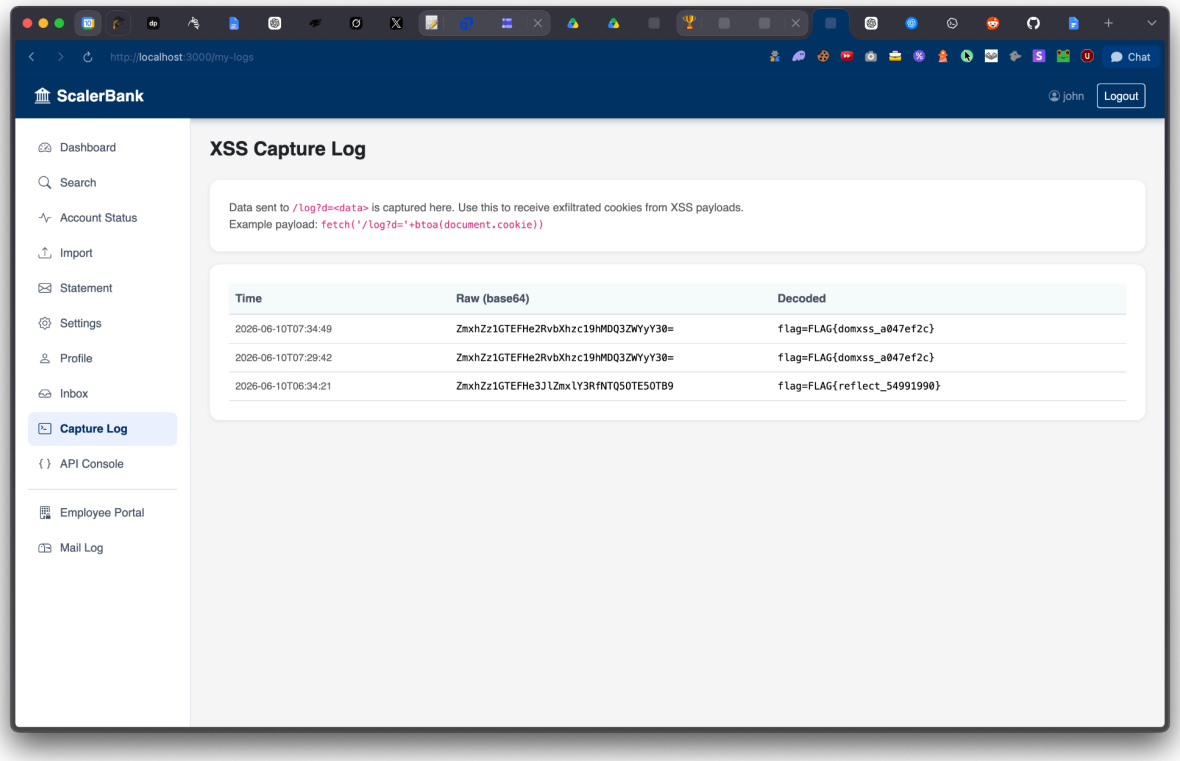
Reflected Cross-Site Scripting (XSS) in the transaction search feature at `/transactions/search`. The search term is rendered in the page using Jinja2's `| safe` filter, which disables HTML escaping. This allows JavaScript injection through the `q` URL parameter. Combined with a report-a-URL feature where an admin bot visits submitted links, an attacker can execute JavaScript in the admin's browser session and steal session cookies.

Exploitation Steps/Description

1. Logged in as john and went to the transaction search page. Typed a normal search like "Salary" and noticed the results page showed: *Showing results for: Salary*. The search term was echoed back.
2. The hint said "your search is not only for you; think about who else might see the same result." Combined with the report form on the same page — where you submit a URL for staff review, it clicked. This was a reflected XSS where the admin bot would see my crafted search.
3. Tried injecting HTML in the search: searched for `<b>test</b>` and it rendered as bold text instead of showing the tag. Confirmed no escaping.
4. Crafted a search URL that would steal the admin's cookies when they visited: http://localhost:3000/transactions/search?q=<img src=x onerror="fetch('/log?d='+btoa(document.cookie))">. The payload creates a broken image (`src=x` fails to load), which triggers the onerror handler, a JavaScript snippet that reads document.cookie and sends it (base64-encoded) to the app's `/log` endpoint.
5. Submitted this URL through the report form on the same page. Waited about 10 seconds for the bot to pick it up.
6. Checked the **Capture Log** page from the sidebar. There was a new entry with base64-encoded data. Decoded it and got the admin's cookies — including the flag cookie.

The flow: searched for something malicious -> the search term became part of the page -> reported the URL -> the admin bot visited -> the malicious code ran -> cookies got shipped to the log -> I read the log.

Screenshots / Evidence



Flag Obtained - FLAG{reflect_54991990}`

Impact

In a real bank, an attacker sends a phishing email with a link to a "transaction search result." When the employee clicks it, the XSS fires and steals their session cookie. The attacker now has full access as that employee — customer data, transaction approvals, wire transfers. They could also modify what the employee sees on the page, inject fake forms to steal credentials, or silently make requests to internal APIs. Reflected XSS is the gateway to account takeover.

Mitigation / Remediation

Never mark user input as "safe" in templates. The `| safe` filter in the HTML template tells the framework to skip escaping — just remove it. Without that flag, `<script>` tags show up as harmless text instead of executable code. Also add a Content Security Policy header so even if escaping fails somewhere, the browser blocks inline scripts.

Import Statement - Challenge 4

Vulnerability Identified

XML External Entity (XXE) injection in the bulk transaction import preview feature at `/tools/import`. The XML parser is configured with `resolve_entities=True`, allowing external entities to be defined and resolved. This lets an attacker read arbitrary files from the server's filesystem by referencing them in the XML document.

Steps

1. The page has a textarea where you can paste raw XML for preview. I pasted this:

```
<?xml version="1.0"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///app/data/flag_xxe.txt">
]>
<transactions>
  <transaction>
    <ref>&xxe;</ref>
    <amount>100</amount>
  </transaction>
</transactions>
```

2. Clicked Preview Import. The preview table showed the file contents in the Reference column. The flag appeared right there in the UI.

What happened: The `<!ENTITY>` declaration tells the XML parser "when you see `&xxe;`, go read this file and put its contents here." Since the parser has `resolve_entities=True`, it obediently reads the flag file and renders the contents in the preview table.

Flag Obtained - FLAG{xxe_700161b1}

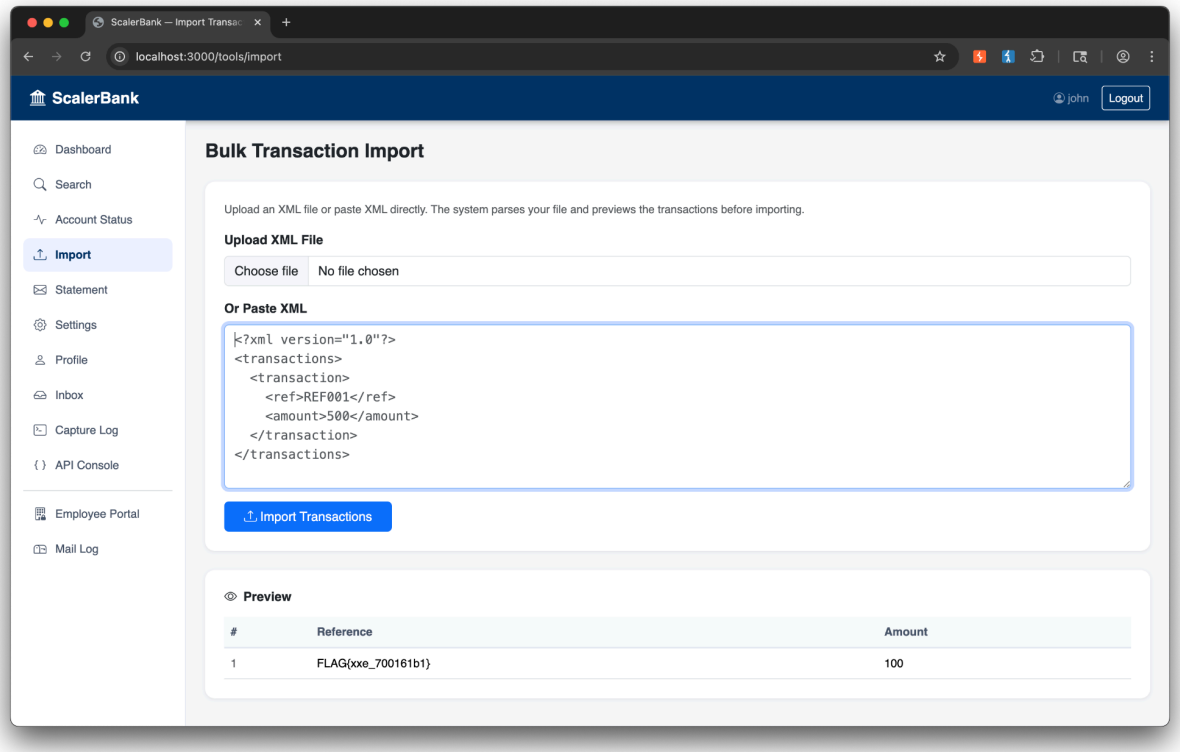
Impact

In a real bank, this would let someone read /etc/passwd, application config files with database passwords, API keys in environment files, private SSH keys, and any sensitive document stored on the server. Essentially a full filesystem read from a feature that was supposed to just preview XML.

Mitigation / Remediation

Disable external entity resolution in the XML parser. With xml, just change resolve_entities=True to resolve_entities=False. Better yet, don't parse user-supplied XML at all, we can use a safer format like JSON for import previews, or use defusedxml which blocks these attacks by default.

Screenshot / Evidence



Statement Mailer - Challenge 5

Vulnerability Identified

OS Command Injection in the account statement request form at `/tools/statement`. The email address field is directly concatenated into a shell command executed with `shell=True`, without any input sanitization or validation. This allows an attacker to inject arbitrary shell commands that execute on the server.

Exploitation Steps/Description

1. Logged in as john and went to the Statement page.
2. The form asks for an account ID and an email address. The hint said "the email field is only the beginning" — so I figured the email was being passed to some backend process.
3. Instead of a real email, I typed: x; cat /app/data/flag_cmdi.txt, the semicolon ends the intended `sendmail` command, and `cat` reads the flag file. The output of both commands shows up in the response on the page.
4. Hit submit. The flag appeared in the output box right below the form.

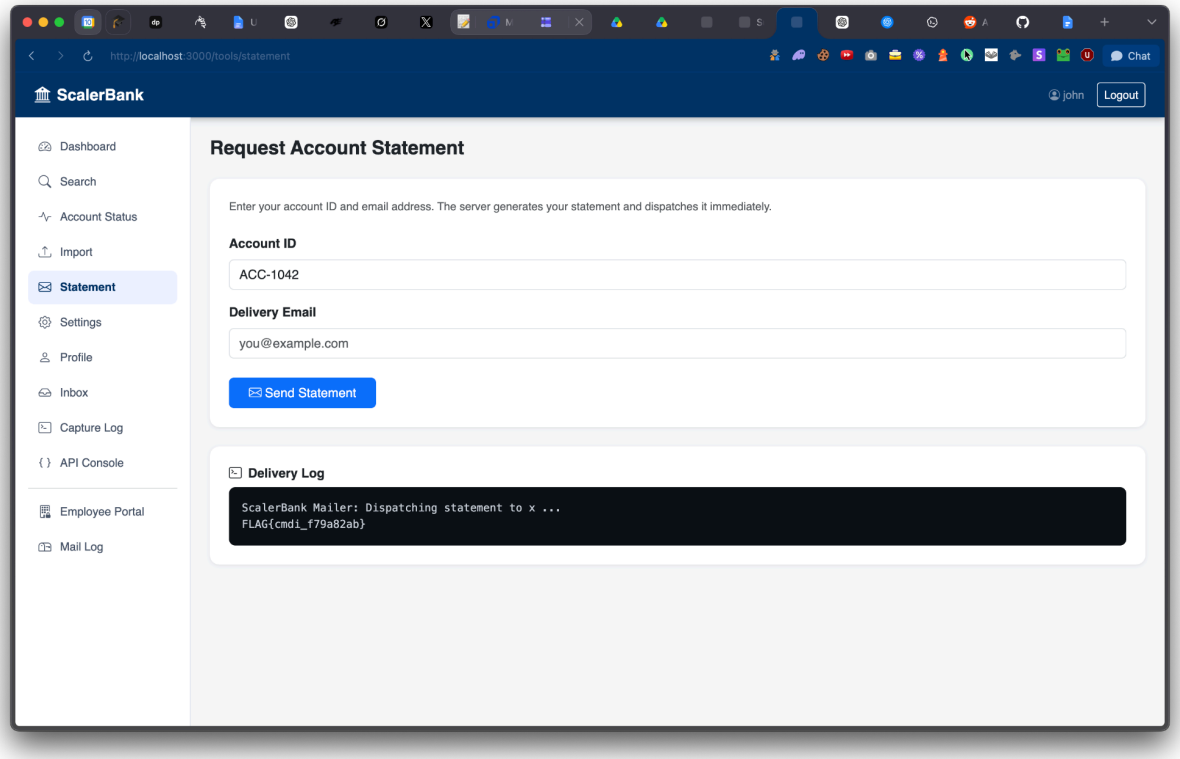
The server essentially ran: `sendmail x; cat /app/data/flag_cmdi.txt` — the first part fails silently, the second part prints the flag.

Flag Obtained - FLAG{cmdi_f79a82ab}

Impact

In a real bank, this is catastrophic. An attacker with a regular customer account could read any file on the server (configs, database backups, private keys), delete or modify data, install backdoors or malware, pivot to other internal systems from the compromised server, and exfiltrate customer data in bulk. All from a feature that was supposed to just email a statement.

Mitigation / Remediation

Never pass user input to `shell=True`. Use `subprocess.run()` with a list of arguments instead of a string — this way the shell never interprets special characters like `;` or `|`. For the email field specifically, validate it against a strict email regex so commands can't sneak in. Run the application process with minimal OS permissions so even if someone breaks out, they can't do much damage.

Screenshot / Evidence